Research Intake 2026

Day 34

Generative Models - GANs

A Beginner's Guide for Students

Learning Computer Vision Through Images, Videos, and Artificial Intelligence

Gudsky Research Foundation

Table of Contents

The future of AI is bright, and you can be part of it!

1 From Discriminative to Generative

Day 33 §10.4 closed with a module-arc framing note worth repeating exactly, since it is the hinge this entire day turns on: Days 26 through 33 have all been discriminative tasks — each maps a given input image to a label, a box, or a mask, and in every case a fixed, objectively correct answer exists for a given input, with the network's job being to find it. This day turns that around entirely. Rather than labeling an existing image, a generative model is asked to produce a new image that did not exist before — no input image to map from at all, only a source of randomness to shape into something that looks like it could have come from the training distribution.

It is worth being precise about why this is a fundamentally different objective, not merely a harder version of the same one. Every discriminative task this module has covered has a well-defined notion of correctness that a loss function can directly measure — a predicted label either matches the ground-truth label or it does not (Day 29), a predicted box either has high IoU with a ground-truth box or it does not (Day 31), a predicted mask either overlaps a ground-truth mask or it does not (Day 33). Generative modeling has no ground-truth output to compare against at all: there is no single correct image a generator should have produced for a given noise input, only a broad question of whether the image it did produce looks like it plausibly belongs to the same distribution as real training images. This absence of a direct target is precisely why Section 9's evaluation methodology looks so different from every evaluation metric this compendium has used so far, and why Section 3's training dynamics are so much less stable than the supervised training Days 29 through 33 relied on.

This day covers one specific, historically dominant approach to generative modeling: Generative Adversarial Networks (GANs), which frame generation as a competitive game between two networks rather than a single network optimizing a fixed target. Day 35, immediately following, covers diffusion models — a genuinely different generative paradigm solving the same underlying problem through a different mechanism entirely, and Section 10.1 previews that comparison directly. It is worth flagging this forward reference now: this day should not be read as "the" way to do generative modeling, only as the first of two approaches this module develops, with Day 35 completing the picture.

> **Callout Box:** *Before the mechanics, it is worth making GANs' capability concrete. A well-trained GAN can produce photorealistic human faces that do not belong to any real person — synthetic portraits detailed enough that distinguishing them from genuine photographs, at a glance, is genuinely difficult even for attentive human viewers. This is the capability the rest of this day explains the mechanism behind, and it is also precisely the capability Section 8's cultural-impact discussion and Section 10.2's ethics section return to: the same technical achievement that makes GANs a genuinely impressive generative tool is what makes synthetic media detection (Section 10.2) a serious, practical concern.*

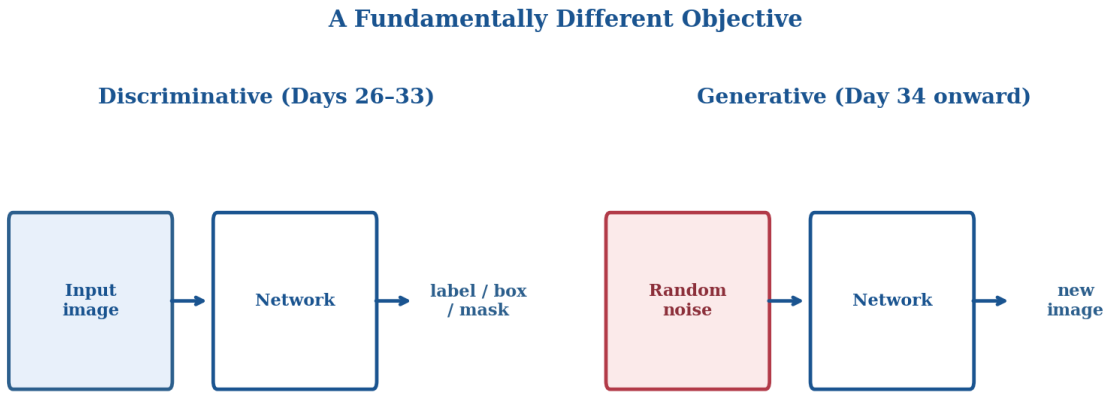

Figure 1. *Every architecture Days 26–33 covered maps an input image to a label, box, or mask. A generative model instead maps random noise to a new image — no ground-truth output exists to compare against, which is the root cause of Section 9's very different evaluation approach.*

2 The GAN Framework

Generative Adversarial Networks (Goodfellow et al., 2014) frame generation as a game between two separately trained networks with opposing objectives, illustrated in Figure 2. The generator takes random noise as input and attempts to produce an image realistic enough to pass as genuine training data. The discriminator takes an image — either a real training image or one of the generator's fabrications — and attempts to correctly classify which of the two it is. The two networks train together, each one's improvement forcing the other to improve in response: as the discriminator gets better at spotting fakes, the generator is pushed to produce more convincing ones, and as the generator's fakes get more convincing, the discriminator is pushed to develop a sharper notion of what distinguishes real images from generated ones.

The counterfeiter-and-detective analogy in Figure 2's captioning is worth taking seriously as more than a memory aid, since it captures the game's structure precisely. A counterfeiter producing fake currency and a detective trying to identify fakes are locked in exactly this kind of escalating competition: the counterfeiter's only useful feedback is whether a given fake was caught, and improves specifically in response to the detective's current detection ability; the detective's only useful training data is an evolving stream of increasingly sophisticated fakes, and improves specifically in response to the counterfeiter's current skill. Neither party has a fixed, external standard of "good enough" to train against — each one's target is defined entirely by the other's current ability, which is exactly the property that makes the mathematical formulation below, and Section 3's training dynamics, so different from anything Day 29's classification loss involved.

It is worth also being precise about what "realistic" means operationally in this framework, since it is easy to leave implicit. The discriminator never has access to any explicit rule defining realism — it learns what counts as real purely from exposure to the training data's own statistics, exactly as Day 29's classifier learned what counts as "cat" purely from labeled examples rather than a hand-coded definition. This means a GAN's notion of realism is entirely bounded by its training data: a face-generating GAN trained exclusively on adult faces has no basis for judging child faces as realistic or not, a limitation worth keeping in mind when interpreting any GAN's output quality as "realistic" in some universal sense rather than realistic specifically relative to its own training distribution.

Formally, this is expressed as a minimax objective: the discriminator tries to maximize its ability to correctly classify real versus fake images, while the generator tries to minimize the discriminator's success rate on its own generated images — a single objective function that one network tries to maximize and the other tries to minimize simultaneously. This adversarial framing was a genuinely novel training paradigm relative to every network Days 29 through 33 trained: those networks were each optimizing a single, fixed loss function toward a single, well-defined target (minimize classification error, maximize IoU with ground truth), while a GAN's generator and discriminator are each chasing a moving target defined by the other network's current state, with no fixed point either one is optimizing toward in isolation.

It is worth being explicit about what "winning" this game actually means, since the minimax framing can otherwise sound like it should resolve into one network simply defeating the other. The theoretical ideal outcome is a Nash equilibrium at which the generator produces samples statistically indistinguishable from real data and the discriminator, faced with genuinely indistinguishable inputs, can do no better than random guessing (fifty percent accuracy) at telling them apart — neither network "wins" in the sense of defeating the other; the game reaches a stable balance in which the generator has become good enough that the discriminator's task is, in principle, impossible to do better than chance at. Section 3 will make clear why real training runs rarely reach this ideal cleanly.

It is worth adding one further structural observation before moving to training dynamics, since it clarifies exactly what makes GANs a genuinely new category of model within this compendium rather than simply a new architecture within the existing supervised paradigm. Every network Days 29 through 33 trained had one loss function and one clear notion of what a fully-trained, converged model looks like. A GAN has two networks, two loss functions defined in terms of each other, and — as this section's Nash equilibrium framing already hints — no single, static endpoint either network converges to in isolation; "trained" for a GAN means something closer to "has reached a productive, stable balance" than "has minimized a fixed loss to its floor," a genuinely different notion of training completion worth carrying forward into every subsequent section of this day.

3 Training Dynamics & Adversarial Loss

3.1 The Original GAN Loss Function

The original GAN formulation trains the discriminator with a standard binary cross-entropy loss — is this image real or fake, the same loss family Day 29's binary classification tasks used — applied to a batch mixing real training images (labeled real) and generator outputs (labeled fake). The generator's loss is defined in terms of the discriminator's output on the generator's own samples: the generator is trained to push the discriminator's predicted "fake" probability on its outputs as low as possible, i.e., to make the discriminator increasingly likely to (incorrectly) call its generated images real.

It is worth being explicit about the training loop's alternating structure, since it differs from every training loop Days 29 through 33 used. A GAN training step typically alternates between updating the discriminator (holding the generator fixed, using a batch of real and generated images) and updating the generator (holding the discriminator fixed, using the discriminator's judgment on a fresh batch of generated images as the training signal) — two separate optimization steps per training iteration rather than Day 29's single forward-backward pass through one network, and a structural detail that itself needs to be implemented correctly, since freezing the wrong network's parameters during either update step is a common, easy-to-make implementation error with no explicit error message to flag it.

3.2 Why GANs Are Notoriously Hard to Train

Every classification and detection loss this compendium has used up through Day 33 shares one property GAN training loses entirely: the loss value itself is a meaningful, monotonically informative signal of progress — a falling classification loss (Day 29) or a rising mAP (Day 31) reliably indicates the model is improving. A GAN's generator and discriminator losses do not have this property, illustrated in Figure 3: because each network's loss is defined relative to the other network's current, constantly-shifting state, a falling generator loss does not necessarily mean the generator is producing better images — it might simply mean the discriminator has temporarily gotten worse, for reasons that have nothing to do with the generator's own progress. Watching either loss curve in isolation, the way Day 29's training curves could be watched, is actively misleading for a GAN; the two curves have to be read together, and even then, oscillation rather than smooth convergence is the expected, normal pattern rather than a sign of a bug.

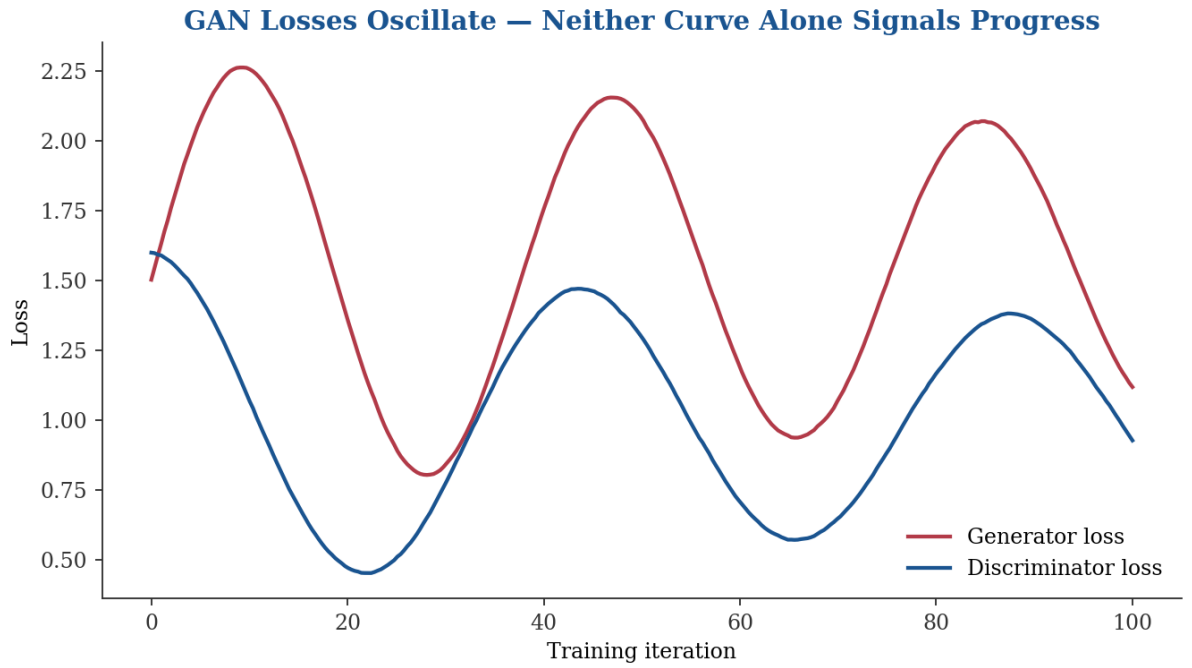

Figure 2. *Unlike Day 29's classification loss or Day 31's detection loss, neither GAN loss curve alone indicates training progress — both oscillate as generator and discriminator alternately gain and lose ground against each other, which is the expected pattern, not a sign of a bug.*

This lack of a single, trustworthy scalar progress signal is precisely why Section 9's evaluation metrics (FID in particular) exist as an external check on training progress, and why Section 4's failure-mode diagnostics rely so heavily on directly inspecting generated samples rather than reading loss curves alone — a recurring piece of practical advice worth internalizing before PP14's own training runs: watching generated samples periodically throughout training is a more reliable progress signal than either loss curve by itself.

3.3 Balancing Generator and Discriminator Training

A further practical complication follows directly from the adversarial framing: if either network becomes too strong relative to the other too early in training, the weaker network's learning signal degrades. A discriminator that becomes near-perfect at distinguishing real from fake images provides the generator with almost no useful gradient — every generated sample is confidently and correctly labeled fake, giving the generator little information about which direction would make its next attempt more convincing. Section 3.4's non-saturating loss is a direct response to exactly this problem. Balancing the two networks' relative training speed (for instance, updating the generator more frequently than the discriminator, or vice versa, depending on which network appears to be pulling ahead) is a standard, hands-on part of GAN training that has no real analogue in Day 29 through Day 33's single-network training loops.

It is worth naming why this balancing act has no clean, universal formula and instead remains a genuinely empirical, per-dataset judgment call. The right relative training speed between generator and discriminator depends on factors specific to a given dataset and architecture — how quickly the discriminator's task

saturates, how expressive the generator's architecture is relative to the discriminator's — none of which can be determined in advance without simply training and watching the outcome. This is a meaningfully different situation from Day 29's learning-rate tuning, which at least has a single network's loss curve as a comparatively reliable guide; balancing two interacting networks' relative training speed requires the kind of sample-quality-driven judgment Section 3.2 already recommended over loss-curve reading alone.

3.4 Non-Saturating Loss: A Practical Fix for Vanishing Gradients

The original minimax generator loss (minimize the discriminator's confidence that generated images are fake) has a specific, well-documented weakness early in training: when the discriminator is confidently correct (as it typically is early on, before the generator has learned anything useful), the original loss's gradient with respect to the generator's parameters is very small — precisely the vanishing-gradient problem Day 30 §3.1 described in a different context (very deep plain networks), recurring here for a structurally different reason. The non-saturating loss reformulates the generator's objective — instead of minimizing the discriminator's confidence that its samples are fake, the generator directly maximizes the discriminator's confidence that its samples are real — a mathematically related but practically much better-behaved objective, since it produces a strong, useful gradient specifically in the regime (early training, weak generator) where the original formulation's gradient is weakest. This reformulation is now the standard practical choice for GAN training, rather than a niche variant.

4 Common GAN Failure Modes

This section matters practically, not merely theoretically: given Section 3's genuinely difficult training dynamics, PP14's own training runs will very likely encounter at least one of the failure modes below, and recognizing which one is occurring — rather than assuming a generic bug — is directly useful for the assignment's tips and pitfalls (Section E of the assignment brief).

It is worth framing why this section exists as its own dedicated part of the day, distinct from Section 3's general training-dynamics discussion. Section 3 explained why GAN training is inherently harder to read than Day 29 through Day 33's supervised training loops; this section catalogs the specific, recognizable ways that difficulty manifests in practice, precisely so that a concerning training run can be matched against a known pattern rather than treated as an undifferentiated, unexplained failure. Each of the three failure modes below has a distinct cause, a distinct diagnostic signature, and a distinct standard response — treating them as a single undifferentiated "GAN training is unstable" phenomenon obscures exactly the diagnostic distinctions PP14's own debugging will depend on.

4.1 Mode Collapse

Mode collapse occurs when the generator discovers a small number of outputs (in the extreme case, a single output) that reliably fool the current discriminator, and collapses onto producing near-identical variations of those few outputs rather than the full diversity of the true training distribution, illustrated in Figure 4. This is a direct, if perverse, consequence of the adversarial objective as stated: the generator's loss only rewards fooling the discriminator, with no explicit term rewarding diversity across the batch of samples it produces, so a generator that has found even one reliably convincing output has, from the loss function's narrow perspective, no explicit incentive to keep exploring for others.

It is worth naming why mode collapse is specifically a GAN problem rather than a generic deep-learning training issue, since the distinction clarifies why none of Days 29 through 33's networks ever exhibited anything analogous. A classification network (Day 29) is trained against a fixed, externally-defined target for every input, so there is no mechanism by which it could "collapse" onto producing the same output regardless of input — its loss function directly penalizes exactly that. A GAN's generator has no such fixed per-input target at all, only the discriminator's current, ever-shifting judgment, which is precisely the structural gap mode collapse exploits.

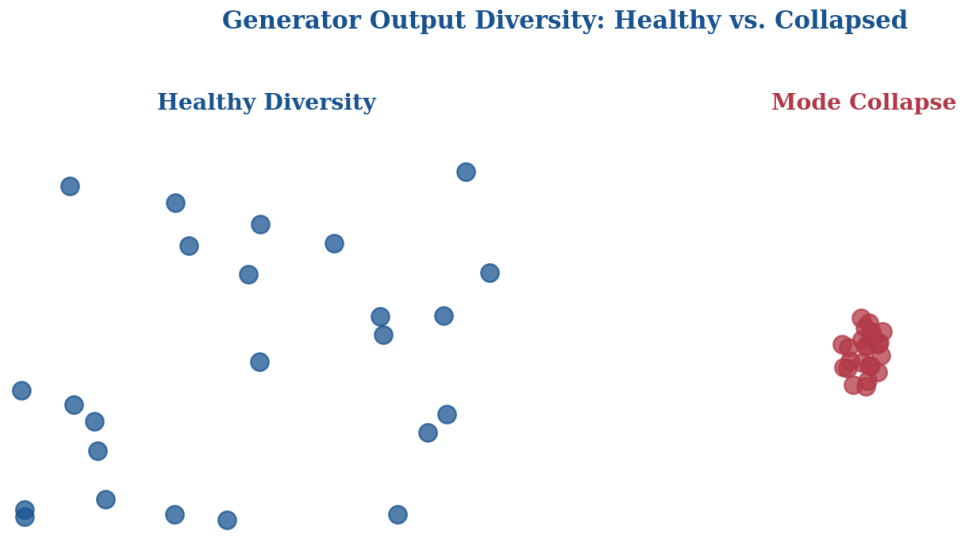

Figure 3. *Healthy generator output spans the training distribution's diversity; a mode-collapsed generator produces near-identical samples clustered around whatever narrow output currently fools the discriminator — a direct consequence of the adversarial loss having no explicit diversity term.*

Diagnosing mode collapse in practice is comparatively straightforward once it is recognized as the specific failure it is, which is precisely why Section 3.2's advice to inspect samples directly, not just loss curves, matters here specifically: a batch of generated images that all look nearly identical, despite different random noise inputs, is the direct visual signature of mode collapse, and no loss-curve reading alone reliably reveals it — the discriminator and generator losses can both look unremarkable while the generator has already collapsed onto a narrow set of outputs.

4.2 Training Instability and Oscillation

Beyond mode collapse, GAN training can exhibit persistent oscillation without ever settling into Section 2's Nash-equilibrium ideal — the generator and discriminator can cycle through a sequence of temporary advantages without converging, each network's improvement destabilizing whatever balance the previous training step had reached. This is distinct from Figure 3's expected, normal loss oscillation: genuine training instability shows up as generated sample quality itself oscillating or degrading over the course of training, not merely the loss values moving up and down while sample quality steadily improves underneath.

It is worth being specific about a further, subtler variant of this instability worth watching for: catastrophic forgetting within a single training run, where the generator, having learned to produce a particular kind of convincing output at one point in training, later loses that capability as the discriminator's continued adaptation pushes the generator's parameters somewhere new — a visible regression in sample quality partway through an otherwise apparently healthy training run, distinguishable from simple noise in the loss curves precisely because it shows up as a genuine, sustained drop in generated sample quality over a meaningful stretch of iterations, not a single noisy fluctuation.

4.3 Vanishing Gradients When the Discriminator Is Too Strong

Section 3.3 already introduced this failure mode's cause; it is worth naming its diagnostic signature explicitly here alongside mode collapse and instability. When the discriminator overpowers the generator, generated sample quality typically plateaus early and does not meaningfully improve for many subsequent training iterations, even as the discriminator's own loss continues to fall — a discriminator that has become too good too fast is providing the generator essentially no useful learning signal, and Section 3.4's non-saturating loss, or deliberately slowing the discriminator's training relative to the generator's (Section 3.3), are the two standard practical responses.

It is worth naming a specific quantity worth monitoring directly, beyond simply eyeballing generated samples, to catch this failure mode early: the discriminator's average confidence (predicted probability) on the generator's current outputs. If this value sits persistently near zero — meaning the discriminator is essentially certain every generated sample is fake, with high confidence, across many consecutive training iterations — this is a direct, quantitative signal of exactly the overpowered-discriminator regime this subsection describes, catchable well before the qualitative sample-quality plateau becomes obvious to the eye.

It is worth summarizing all three failure modes' diagnostic signatures side by side, since distinguishing them in practice is precisely what determines which fix (Sections 3.3–3.4, or the DCGAN guidelines in Section 5) is the relevant one to apply. Mode collapse shows as low output diversity despite varied noise inputs. Instability shows as generated sample quality itself fluctuating or degrading over training, not merely the loss curves. An overpowered discriminator shows as generated sample quality plateauing early

while the discriminator's own loss keeps improving. Misapplying one failure mode's fix to a different actual problem — for instance, slowing discriminator training when the real issue is mode collapse — wastes training time without addressing the underlying cause, which is exactly why correct diagnosis matters before reaching for a fix.

5 DCGAN & Architectural Guidelines

DCGAN (Deep Convolutional GAN, Radford et al., 2015) did not change the adversarial training framework Sections 2 and 3 developed; instead, it identified a specific set of architectural guidelines that made the original GAN idea substantially more stable and practical to train — the architectural stabilization of an already-sound training idea, in much the same spirit as Day 30 §3's residual connections stabilizing an already-sound "deeper is better" idea for classification networks.

It is worth being clear about DCGAN's historical significance before its individual guidelines, since it is easy to undersell. Before DCGAN, GAN training was widely regarded as unreliable enough to be a serious practical obstacle to using the idea at all — Section 3 and Section 4's difficulties were, in the field's earliest years, closer to the rule than the exception. DCGAN's contribution was not a new training algorithm or loss function, but a demonstration that a specific, disciplined set of architectural choices could make the existing minimax framework (Section 2) reliably trainable in practice, which is precisely why its guidelines became a starting-point template rather than one competing option among many equally viable alternatives.

5.1 Strided Convolutions Instead of Pooling

Day 29 §4 covered pooling as a standard, parameter-free downsampling operation for classification networks. DCGAN's guidelines replace pooling entirely, in both the generator and discriminator, with strided convolutions (in the discriminator, for downsampling) and strided transposed convolutions (in the generator, for upsampling, illustrated in Figure 5). This matters specifically for GAN training because pooling discards information through a fixed, non-learned operation, while a strided convolution's downsampling or upsampling behavior is itself learned during training — giving the network more flexibility to learn spatial transformations suited to the generative task, rather than being constrained by pooling's fixed, hand-designed downsampling rule.

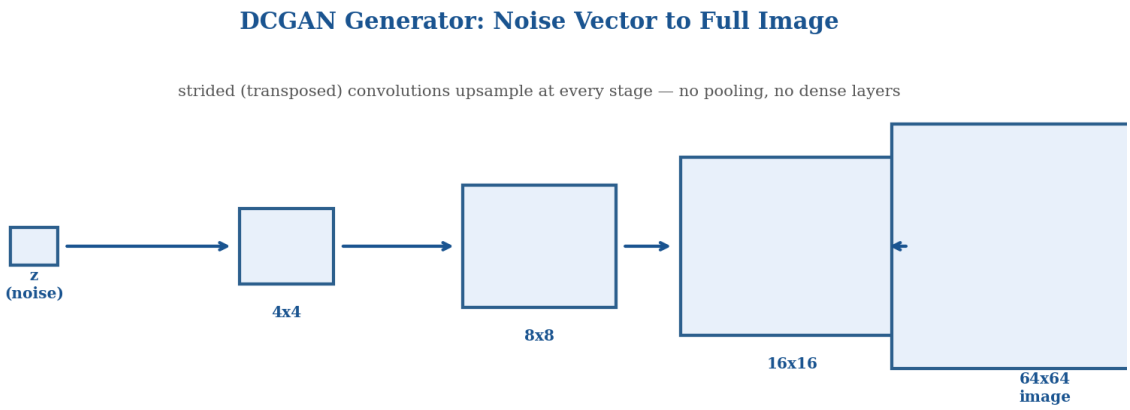

Figure 4. *DCGAN's generator maps a low-dimensional noise vector through a series of strided transposed convolutions, progressively upsampling to full image resolution — no pooling, no fully-connected layers, exactly the guidelines this section develops.*

5.2 Batch Normalization in Both Networks

Day 29 §8 introduced batch normalization as a technique for stabilizing training by normalizing layer activations, and DCGAN applies it in both the generator and the discriminator (with specific exceptions at the generator's output layer and the discriminator's input layer, where normalization is typically omitted). This is particularly valuable for GAN training given Section 3's already-difficult dynamics: batch normalization helps prevent the kind of extreme activation values that can push either network toward the vanishing-gradient regime Section 3.4 addressed at the loss-function level, providing a complementary, architecture-level stabilization on top of the loss-function-level fix.

5.3 Avoiding Fully-Connected Layers

DCGAN's guidelines recommend avoiding fully-connected hidden layers almost entirely, keeping the network convolutional throughout except at the very first layer of the generator (which must expand a low-dimensional noise vector into a small spatial feature map to begin the upsampling process) and, in some formulations, the discriminator's final classification layer. This echoes Day 33 §3.1's FCN insight from a different direction: where Day 33's FCN removed fully-connected layers to preserve spatial structure for dense prediction, DCGAN removes them to preserve spatial structure throughout a generative network's upsampling path, for a related but distinct reason — a fully-connected layer's lack of spatial structure is precisely what a generator's progressively-upsampling design needs to avoid disrupting at every stage.

It is worth being precise about the practical consequence of retaining even one fully-connected layer, since it is a subtle enough point that a first attempt at implementing DCGAN's guidelines can easily miss it. The generator's very first layer necessarily has to convert a flat, unstructured noise vector into some initial spatial arrangement — there is no way to apply a convolution to a vector with no spatial dimensions at all — but every layer after that first reshape should remain convolutional throughout the entire upsampling path (Figure 5). A generator design that reintroduces a fully-connected layer partway through its upsampling

path, rather than only at this unavoidable first step, discards spatial structure the preceding convolutional layers had already built up, reintroducing exactly the problem DCGAN's guidelines were designed to eliminate.

5.4 Activation Function Choices and Why These Guidelines Became a Standard Template

DCGAN's guidelines also specify particular activation function choices: ReLU activations throughout the generator (except a tanh activation at the final output layer, matching the normalized pixel-value range training images are typically scaled to) and LeakyReLU activations throughout the discriminator, chosen specifically because LeakyReLU's small negative-region gradient (rather than ReLU's hard zero) helps gradient flow through the discriminator during exactly the phases of training where gradients are already fragile, per Section 3.2's broader training-instability discussion.

Taken together, these guidelines — strided convolutions, batch normalization, no fully-connected layers, and this specific activation choice pairing — became the practical default starting template for nearly every GAN architecture that followed, including the conditional and image-to-image architectures Sections 6 and 7 develop, and are worth treating as a genuine starting point for PP14's Approach A implementation specifically, since they represent the field's converged, empirically validated answer to Section 3's training-stability problem, rather than one option among many equally reasonable choices.

6 Conditional GANs

Every architecture Sections 2 through 5 developed produces unconditional generation: the generator takes only random noise as input, with no control over what specific kind of image it produces beyond whatever the training data's overall distribution happens to favor. A Conditional GAN (cGAN) adds exactly this missing control, illustrated in Figure 6: both the generator and the discriminator receive an additional conditioning input — commonly a class label, but in principle any auxiliary information — alongside their usual inputs, letting a user specify what should be generated rather than only sampling from the training distribution's overall shape.

It is worth grounding this with a concrete before-and-after comparison. An unconditional GAN trained on a dataset of handwritten digits (Approach A's kind of setup) can only be sampled from — running the generator repeatedly and hoping a desired digit happens to appear among the results, with no way to request a specific one. A conditional GAN trained on the same data, with digit label as the condition, can be directly asked to generate a "7": the same underlying generative capability, but now steerable rather than purely stochastic, which is precisely the practical difference PP14's Approach B is designed to make directly observable against Approach A's baseline.

It is worth being precise about the architectural change this requires in both networks, not just the generator. The generator concatenates (or otherwise combines) the noise vector with the conditioning information before beginning its upsampling path, so that the condition influences every subsequent layer's computation. The discriminator, correspondingly, must also receive the same conditioning information alongside the image it is judging, and must learn a stricter notion of "real" than the unconditional case required: an image is now only genuinely real relative to its claimed condition, so a real cat image paired with the label "dog" should be judged fake by a correctly-trained conditional discriminator, exactly as much as a generated image would be — the discriminator's task has become "is this image both realistic and correctly matched to its label," not merely "is this image realistic."

This controllable generation capability is precisely what PP14's Approach B is built to demonstrate directly against Approach A's unconditional baseline: the same underlying training framework (Sections 2–3) and architectural guidelines (Section 5) apply essentially unchanged, with conditioning as the one meaningful structural addition, letting a direct, controlled comparison isolate exactly what conditioning buys in exchange for its added architectural complexity — sample diversity and control, at the cost of requiring labeled (or otherwise conditioned) training data that unconditional generation does not.

It is worth naming a further, less obvious practical benefit conditioning can provide beyond direct user control, since it connects back to Section 4.1's mode collapse discussion. Conditioning can, in some cases, make mode collapse less severe in practice: because the generator's task is now implicitly broken down into many smaller per-condition subtasks (produce a convincing "3", produce a convincing "7", and so on) rather than one single, undifferentiated generation task, a generator that might have collapsed onto a narrow set of outputs in the unconditional setting has less room to do so once each condition carries its own separate expectation of what a correct, diverse output looks like — not a guaranteed fix, but a structural tendency worth watching for when comparing Approach A and Approach B's relative stability in PP14.

Conditional GANs are also the direct conceptual bridge into Section 7's image-to-image translation, worth flagging explicitly before moving on: image-to-image translation can be understood as a conditional GAN where the conditioning input is not a simple class label but an entire other image — a considerably richer form of conditioning than Section 6 has developed so far, but built on exactly the same "feed the condition into both networks" architectural principle this section established.

7 Image-to-Image Translation

7.1 pix2pix: Paired Image Translation

pix2pix is a conditional GAN, per Section 6's framing, in which the condition is an entire source image rather than a class label, and the generator's task is to translate that source image into a corresponding target image — a sketch into a photo, a daytime scene into a nighttime one, a satellite image into a map. Critically,

pix2pix requires paired training data: for every source image, a corresponding, aligned target image must be available (Figure 7's left panel), so the generator has a concrete example of the correct translation to learn from during training, alongside the adversarial signal from the discriminator.

It is worth being explicit about pix2pix's loss function, since it combines two distinct signals rather than relying on the adversarial loss alone. Alongside the discriminator's adversarial signal, pix2pix adds a direct pixel-level reconstruction loss (typically L1 distance) between the generated translation and the known, paired ground-truth target — a direct supervisory signal Section 2's original unconditional GAN framework has no equivalent of, made possible only because pix2pix's paired data provides an actual target to measure against, unlike Section 2's noise-to-image setting where no such target exists at all. This combination is part of why pix2pix trains more predictably than an unconditional GAN: the reconstruction loss provides a stable, non-adversarial gradient signal throughout training, with the adversarial loss contributing sharper, more realistic fine detail on top of it.

pix2pix's generator uses a U-Net-style architecture — a direct callback to Day 33 §4's U-Net — and the reuse is not superficial: image-to-image translation needs exactly the property Day 33 §4.2 established U-Net's skip connections provide, preserving fine spatial detail from the source image (edges, textures, precise structure) across the network's encoder-decoder bottleneck, so the generated output remains structurally aligned with the input rather than only broadly resembling the right category of image. Where Day 33 used this property to recover precise segmentation boundaries, pix2pix uses the identical mechanism to keep a translated image spatially faithful to its source.

7.2 CycleGAN: Unpaired Translation via Cycle-Consistency

pix2pix's paired-data requirement is a genuine practical limitation: aligned source-target pairs (the same scene, once as a sketch and once as a photo) are often far harder to obtain than two separate, unrelated collections of each type of image. CycleGAN removes this requirement entirely, learning to translate between two image domains (Figure 7's right panel — horses and zebras is the canonical illustrative example) using only two unpaired collections, one per domain, with no example ever pairing a specific horse image to a specific corresponding zebra image.

This is made possible by cycle-consistency loss: CycleGAN trains two generators simultaneously, one translating domain A to domain B and another translating domain B back to domain A, and adds a loss term requiring that translating an image from A to B and then back to A again should recover something close to the original image. This is a genuinely clever unsupervised learning signal, worth appreciating on its own terms: it provides a strong, meaningful training constraint with no paired ground truth required at all, since the constraint is defined entirely in terms of the model's own two directions of translation being mutually consistent with each other, rather than either direction needing to be checked against an external paired example.

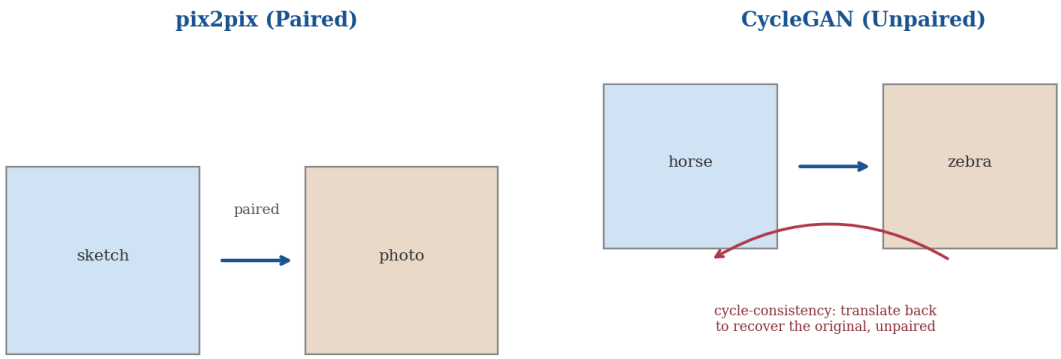

Figure 5. *pix2pix requires paired source-target examples for direct supervision; CycleGAN removes this requirement using cycle-consistency — translating A to B and back to A should recover the original, a training signal that needs no paired data at all.*

> **Research Spotlight:** *Cycle-consistency's "translate and translate back" signal connects to a broader self-supervision theme this compendium has touched on before, in Day 30 §10's preview of self-supervised representation learning: both use a structural consistency constraint the model must satisfy, derived entirely from the model's own outputs, as a training signal that requires no manually labeled ground truth. CycleGAN's specific version of this idea — translate forward, translate back, compare to the original — is a particularly clean, intuitive illustration of self-supervision's general logic, worth keeping in mind as this broader theme recurs again in later modules.*

7.3 Practical Applications

Image-to-image translation's practical applications span style transfer (repainting a photo in a particular artistic style), sketch-to-photo generation (turning a rough hand-drawn sketch into a photorealistic image), and domain adaptation tasks like day-to-night or summer-to-winter scene translation, used both as creative tools and, in some domains, as a form of synthetic data augmentation — translating a dataset collected under one condition (daytime driving footage) into an equivalent under a different, underrepresented condition (nighttime driving footage) rather than needing to collect and label an entirely new dataset from scratch.

It is worth connecting this synthetic-augmentation use case directly back to Day 27 §6's data augmentation discussion, since it represents a genuinely more powerful version of the same underlying idea. Day 27's augmentation techniques (rotation, flipping, color jitter) create new training examples through simple, fixed geometric or photometric transformations of existing images. Image-to-image translation can instead generate entirely new, semantically coherent variations — the same driving scene rendered at night rather than day, with correspondingly different lighting, shadows, and visibility, not merely a color-shifted version of the original daytime image — a qualitatively richer form of augmentation than Day 27's techniques can produce on their own, made possible only because Section 7.1–7.2's translation models have learned an actual mapping between domains rather than applying a fixed, hand-specified transformation.

8 Progressive & High-Resolution GANs

Every architecture Sections 2 through 7 have covered generates comparatively low-resolution images, and it is worth being explicit about why naively scaling any of them up to high resolution does not simply work. Training a GAN directly at high resolution from the very start of training makes Section 3's already-difficult adversarial dynamics substantially worse: at high resolution, the discriminator can typically detect real-versus-fake based on fine, easily-learned local texture details long before the generator has learned any coherent large-scale structure, giving the discriminator an early, overwhelming advantage that pushes training directly into Section 4.3's vanishing-gradient failure mode.

It is worth being specific about why this particular failure is so hard to fix through Section 5's guidelines alone, since it is a genuinely different problem from the training instabilities those guidelines already address. DCGAN's architectural fixes (strided convolutions, batch normalization) make a fixed-resolution GAN more stable at whatever resolution it is trained at, but they do nothing to change the fundamental asymmetry between how quickly a discriminator can learn to exploit fine texture detail versus how slowly a generator can learn to produce coherent large-scale structure — an asymmetry that gets strictly worse as target resolution increases, which is exactly why a training-procedure-level fix, rather than a further architectural tweak, was needed.

8.1 Progressive GAN's Growing-Resolution Strategy

Progressive GAN addresses this directly by changing the training procedure itself, illustrated in Figure 8's left panel: training begins at a very low resolution (a 4x4 image), where both networks can quickly learn coarse, large-scale structure without being distracted by fine texture detail, and new layers are progressively added to both the generator and discriminator over the course of training, gradually increasing the resolution (8x8, then 16x16, and onward) only once the current resolution has stabilized. This staged approach lets both networks master coarse structure before ever having to deal with the fine-detail discrimination problem that made direct high-resolution training so unstable, directly sidestepping the specific failure mode described above rather than trying to out-tune it away with loss-function or architectural fixes alone.

It is worth being specific about how new layers are actually introduced during this process, since an abrupt architectural change partway through training could itself be destabilizing if handled carelessly. Progressive GAN fades new layers in gradually rather than switching them on all at once: a newly added layer's contribution is blended with the previous, lower-resolution output over a transition period, with the new layer's weight increasing smoothly from zero to full strength — avoiding a sudden shock to either network's already-learned weights at the moment resolution increases, and ensuring the growing-resolution strategy adds new capability incrementally rather than introducing a new source of instability at every transition point.

8.2 StyleGAN's Style-Based Generator

StyleGAN builds on Progressive GAN's growing-resolution training strategy but changes the generator's internal architecture substantially: rather than feeding the noise vector directly into the generator's first layer as DCGAN and Progressive GAN both do, StyleGAN first maps the noise vector through a separate mapping network into an intermediate latent space, and then injects style information derived from this intermediate representation at every resolution level of the generator, rather than only at the start.

This design produces a genuinely useful practical property worth naming explicitly: a disentangled latent space, in which different dimensions of the intermediate representation tend to control semantically distinct, separable aspects of the generated image (coarse features like pose and face shape at early-injected layers, finer features like hair texture or skin tone at later-injected layers) rather than every dimension entangling many visual attributes together indiscriminately, as tends to happen when noise is injected only once at the start (DCGAN and Progressive GAN's approach). This disentanglement is what enables the kind of fine-grained, controllable editing (adjusting one visual attribute of a generated face while leaving others largely unchanged) that made StyleGAN's outputs particularly compelling and, correspondingly, particularly consequential for the ethics discussion Section 10.2 develops.

It is worth being precise about why injecting style at every resolution level, rather than only once at the start, is what actually produces this disentanglement, since the mechanism is not obvious from the description alone. Early-injected style information affects only the coarse spatial structure the generator has built up by that point in its upsampling path (Section 8.1's low-resolution stages), while later-injected style information affects only the fine spatial structure available at those later, higher-resolution stages — the architecture itself enforces a rough separation between coarse and fine attributes simply by tying each style injection to a specific stage of the generator's progressive upsampling, rather than relying on the network to discover such a separation unsupervised from a single, undifferentiated noise input.

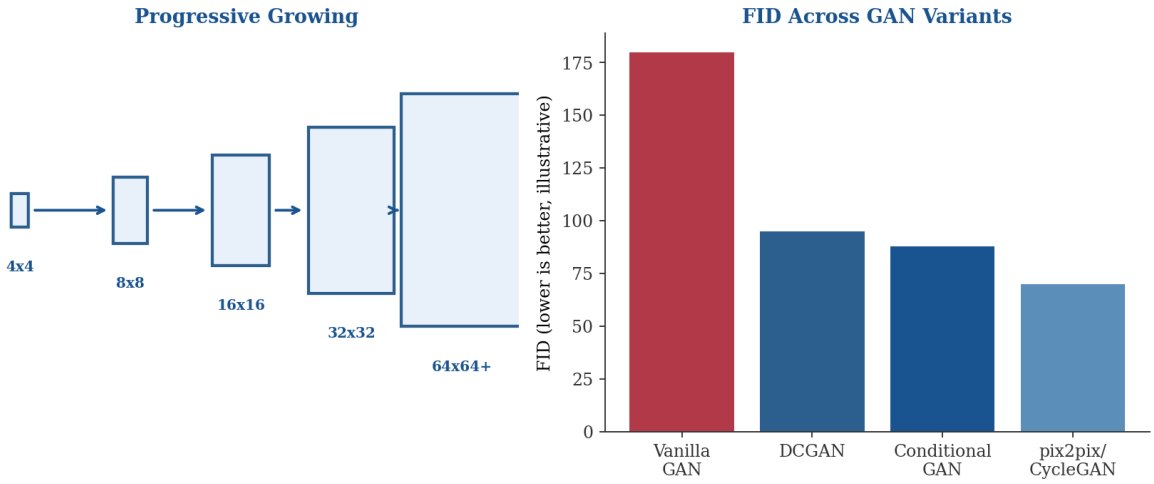

Figure 6. *Progressive GAN trains coarse structure first, adding resolution gradually, directly avoiding the fine-detail discriminator advantage that destabilizes naive high-resolution training. The right panel illustrates FID improving across the GAN variants this day covers — Section 9 develops this metric in full.*

It is worth closing this section with a direct segue into Section 9 and Section 10.2, rather than treating StyleGAN's technical achievement in isolation. StyleGAN-quality synthetic face generation is precisely the technical capability underlying modern deepfake and synthetic-identity concerns — the same disentangled, high-fidelity generation that makes StyleGAN a genuinely impressive research achievement is what makes synthetic media detection (Section 10.2) a serious, practical necessity rather than a hypothetical concern, and it is worth carrying that dual character forward into both the evaluation and ethics sections that follow.

9 Evaluating Generative Models

Section 1 already established the core reason generative evaluation differs fundamentally from every metric this compendium has used through Day 33: there is no ground truth to compare a generated image against, so accuracy (Day 29), mAP (Day 31), and mIoU/Dice (Day 33) are all simply inapplicable here — none of them has any notion of what to compare a generated sample to.

9.1 Inception Score

Inception Score uses a pretrained image classifier (an Inception network, hence the name) to evaluate generated images along two desirable properties simultaneously: each individual generated image should produce a confident, sharply-peaked classification when passed through the classifier (indicating the image contains a clear, recognizable subject rather than an incoherent blur), while the distribution of predicted classes across the full set of generated images should be broad and varied (indicating the generator is producing diverse outputs across many categories, not Section 4.1's mode collapse). Inception Score combines these two properties into a single score, but it has a well-documented weakness worth flagging: it depends on a classifier trained on a fixed category set, and provides no way to directly compare generated images against real training images at all — a generator could score well on Inception Score while still producing images that look nothing like the actual training distribution's specific visual characteristics.

It is worth being specific about a further, more subtle limitation of Inception Score beyond the one stated above, since it directly motivates FID's design. Because Inception Score is computed entirely from the generated images' own classifier outputs, with no reference to real data at all, it cannot detect a generator that has learned to produce sharp, confidently-classified, diverse images that nonetheless look visually nothing like the actual training distribution's style, texture, or composition — a generator could, in principle, score well on Inception Score while producing images a human viewer would immediately recognize as stylistically wrong for the dataset, which is precisely the gap Section 9.2's FID closes by comparing directly against real data.

9.2 Fréchet Inception Distance (FID): The Current Standard

Fréchet Inception Distance addresses Inception Score's central weakness directly: rather than evaluating generated images against a fixed classifier's category confidence alone, FID compares the statistical distribution of deep features (again extracted via a pretrained Inception network, but used here as a feature extractor rather than a classifier) between a set of real images and a set of generated images, computing a distance between the two feature distributions' means and covariances. A lower FID indicates the generated images' feature statistics more closely match the real images' feature statistics — genuinely comparing generated output against real data directly, which is exactly what Inception Score cannot do, and is why FID has become the field's de facto standard reporting metric, referenced throughout Figure 8's right panel and PP14's own evaluation requirements.

It is worth being clear about what FID does and does not require, since it is a meaningfully different kind of metric from every one this compendium has used through Day 33. FID needs no labels at all — not the generated images' intended class, not any ground-truth target — only two unordered collections of images, real and generated, which is precisely why it generalizes cleanly across Section 9.5's three architecturally different PP14 approaches despite their otherwise very different training setups (unconditional, conditional, paired or unpaired translation). This label-free property is a direct consequence of Section 1's core observation that generative evaluation has no ground truth to compare against — FID works around that absence by comparing distributions rather than individual predictions to individual targets, the same conceptual move underlying every generative metric this section covers.

9.3 Qualitative and Human Evaluation

Neither Inception Score nor FID fully captures everything a human viewer would notice about generated image quality — subtle artifacts, semantic implausibility (a face with an anatomically incorrect number of fingers, in an adjacent generative domain), or stylistic mismatches that a feature-distribution comparison can miss entirely. Direct qualitative inspection of generated samples, and in more rigorous settings structured human evaluation (asking human raters to distinguish real from generated images, or to rate generated image quality directly), remains a standard, necessary complement to Section 9.1 and 9.2's automated metrics rather than a lesser, informal substitute for them.

9.4 Precision-Recall for Generative Models: The Diversity-vs-Fidelity Tradeoff

A further refinement worth knowing about adapts Day 31 §3.1's precision/recall framing to the generative setting, though the two concepts mean something structurally different here. Generative precision measures how much of the generated distribution falls within the real data distribution's support (roughly: how realistic individual generated samples are), while generative recall measures how much of the real data distribution's diversity the generator's output distribution actually covers (roughly: how much of the true

variety in the training data the generator has learned to reproduce, directly related to Section 4.1's mode collapse). A generator that has mode-collapsed onto a few highly convincing outputs can score well on generative precision while scoring poorly on generative recall — exactly the diversity-versus-fidelity tradeoff this metric pair is designed to expose, where a single aggregate score like FID could mask the distinction entirely.

It is worth adding a concrete illustration of exactly how FID can mask this distinction, since it is the clearest argument for reporting precision-recall alongside FID rather than FID alone. A severely mode-collapsed generator producing only a handful of extremely convincing outputs can, in some cases, still achieve a deceptively reasonable FID score, since FID compares aggregate feature-distribution statistics rather than checking coverage of the real distribution directly — a narrow but highly realistic cluster of generated samples can pull the aggregate statistics closer to the real distribution's mean than the underlying lack of diversity would suggest. Generative precision-recall, computed to check coverage directly rather than only aggregate statistical proximity, catches exactly this case, which is precisely why it exists as a complement to FID rather than a redundant restatement of it.

9.5 Practical Guidance for PP14's Head-to-Head Comparison

PP14 requires comparing three architecturally different approaches (Sections 5–7's vanilla/DCGAN, conditional, and image-to-image variants) directly against each other, which raises a genuine practical question this subsection addresses directly: FID depends on a reasonably large sample of generated and real images to produce a statistically stable estimate, and computing it reliably for all three approaches within a single Colab T4 session may not always be practical given time constraints, particularly for Approach C's image-to-image setting, where "real" and "generated" comparison sets are less straightforward to define than in Section 9.1–9.2's unconditional setting. Where FID is impractical within the assignment's compute budget, structured qualitative comparison — a consistent grid of generated samples per approach, evaluated side by side against the same criteria (realism, diversity, and, for conditional and translation approaches, correctness relative to the given condition) — is an entirely reasonable and expected substitute, provided the comparison is applied consistently across all three approaches rather than different evaluation standards for each.

10 Research Frontiers

10.1 GANs vs. Diffusion Models: A Forward Reference to Day 35

This day has developed one complete generative paradigm — an adversarial game between two competing networks — in full. Day 35, immediately following, develops diffusion models: a genuinely different generative mechanism, learning to reverse a gradual noising process rather than training two networks against each other. It is worth previewing the shape of that comparison now, since it completes this

generative arc rather than leaving GANs as an isolated, self-contained topic. GANs, as this day has shown, are notoriously difficult to train (Section 3) and prone to specific, well-characterized failure modes (Section 4) but are comparatively fast at generation once trained, producing a sample in a single forward pass through the generator. Diffusion models, Day 35 will develop, tend to train more stably (no adversarial game, no Section 4 failure modes to diagnose) but are typically slower at generation, requiring many sequential steps to produce a single sample rather than one forward pass. Day 35 §8 develops this speed-versus-stability tradeoff in full, once diffusion's own mechanics have been established — this day's job is only to set up the comparison's first half.

It is worth being fair to GANs specifically before moving to the ethics discussion, since the diffusion-versus-GAN framing above could otherwise read as GANs simply being the inferior, superseded option. GANs' single-forward-pass generation speed remains a genuine, practical advantage diffusion models have historically struggled to match, and StyleGAN-family architectures (Section 8.2) remain highly competitive on image quality specifically for the domains they were designed for (face generation prominently among them) even years after diffusion models became widely available — this day's content is not a historical detour ahead of Day 35's "real" answer, but a genuinely competitive, still-actively-used approach in its own right, exactly the "no universal winner, only a fit for constraints" framing this compendium has applied consistently to every architectural comparison since Day 30 §9.

10.2 Ethical Considerations: Deepfakes, Detection, and Consent

Section 8's closing note on StyleGAN's cultural impact is worth developing in full here, directly extending Day 26 §9's bias-and-ethics thread into a genuinely new form of harm this compendium has not yet addressed. Every ethics discussion through Day 33 concerned a model's decisions about real people or real scenes — a biased classifier, a surveillance system's demographic fairness. Generative models introduce a different kind of harm entirely: the creation of synthetic media depicting real individuals without their knowledge or consent (deepfakes), or the generation of entirely fabricated but photorealistic identities used for deception (synthetic-identity fraud, fake social media personas). This is not a harm of the model getting something about a real person wrong, as Day 26 and Day 31's surveillance discussions concerned, but a harm of the model fabricating something about a real or fictional person that was never true at all — a qualitatively different concern that deserves its own explicit consideration rather than being treated as a variant of earlier days' bias discussions.

Synthetic media detection — building classifiers specifically trained to distinguish GAN-generated images from genuine photographs — has emerged as a direct, necessary response to this capability, and it is worth noting the adversarial dynamic this creates one level up from Section 2's original generator-discriminator game: as detection methods improve, generative methods are pushed to produce even more convincing fakes to evade them, an ongoing arms race structurally similar to Section 2's own adversarial training loop, just playing out at the level of published research and deployed detection systems rather than within a single

training run. Consent and disclosure — clearly labeling synthetic media as such, and never generating identifiable depictions of real individuals without their explicit permission — are the baseline responsible-use principles worth carrying forward from this section into any generative application, echoing Day 26 §9.3's responsible-deployment framework in a genuinely new context.

It is worth being explicit about how this responsibility applies directly to PP14 itself, not only to hypothetical future deployments. Even at PP14's compact, single-session scale, Approach B and Approach C's conditional and translation-based generation are capable, in principle, of producing convincing manipulations of specific real individuals if pointed at the wrong training data — a compact GAN trained on a small dataset is not inherently safer than a large one, only smaller in scope. Choosing training data that does not include identifiable real individuals without their consent is a straightforward, practical way to keep the assignment itself squarely on the right side of this section's principles, worth deciding deliberately when selecting Approach B and Approach C's datasets rather than treated as an afterthought.

10.3 GAN Applications Beyond Images

While this day has focused entirely on image generation, it is worth briefly noting that the adversarial training framework Sections 2 and 3 developed is not inherently image-specific — GANs have been applied to audio generation, tabular data synthesis (generating realistic synthetic records for privacy-preserving data sharing or augmenting scarce training data), and molecular structure generation in drug discovery, among other domains, all built on the same generator-discriminator game this day developed, with domain-specific architectural changes replacing Section 5's convolutional guidelines where the underlying data is not itself spatially structured like an image.

10.4 Closing Perspective: The Generative Arc, Continued

Day 33 §10.4 marked this day as the start of the module's generative arc; this day has developed that arc's first half in full — a network learning to produce convincing new images by competing against a second network trained to catch it, from the basic minimax framework (Section 2) through the practical stabilization techniques (Sections 3–5) and creative applications (Sections 6–8) that made the idea genuinely useful rather than merely theoretically elegant. PP14's three-way comparison across unconditional, conditional, and translation-based generation is where this day's theory becomes a measured, personal finding, exactly as Day 31 §8.4 and Day 32's PP15 framed their own comparisons earlier in this module. Day 35 completes the generative arc with a structurally different answer to the same underlying problem, and Section 10.1's preview is worth carrying forward directly into that day's opening framing.

11 Common Pitfalls: A Practical Debugging Checklist

Because this day combines multiple architectural stages and a dense evaluation methodology, a poor result can trace back to any of several places. The following checks catch the most common issues.

- Inspect generated samples directly and frequently during training, not just the loss curves — per Section 3.2, neither GAN loss alone is a trustworthy progress signal, and mode collapse (Section 4.1) in particular can be invisible in the loss curves while being immediately obvious in a grid of generated samples.
- Before concluding training has failed, correctly diagnose which of Section 4's three failure modes is actually occurring — low output diversity signals mode collapse, oscillating or degrading sample quality signals instability, and an early quality plateau alongside a still-improving discriminator loss signals an overpowered discriminator; applying the wrong fix wastes training time without addressing the real problem.
- Verify the non-saturating generator loss (Section 3.4) is actually being used, not the original minimax formulation — the difference is a small, easy-to-miss implementation detail with a large practical effect on early-training gradient quality, and is one of the first things worth checking when a generator's samples are not improving at all in early training.
- Confirm batch normalization is applied consistently per Section 5.2's guidelines (including the specific layers where it is conventionally omitted) — a mismatched normalization setup between generator and discriminator is a common, silent source of the exact training instability Section 4.2 describes.
- For CycleGAN specifically, watch for a poorly-tuned cycle-consistency loss weight — too low, and the model drifts away from Section 7.2's intended consistency constraint entirely, producing translations unrelated to the input; too high, and the model becomes overly conservative, producing translations too close to a simple identity mapping to be useful.
- Watch for discriminator overfitting on a small PP14-scale dataset — with a compact, Colab-appropriate dataset, the discriminator can memorize the specific real training images rather than learning generalizable real-versus-fake features, pushing training toward Section 4.3's vanishing-gradient regime faster than it would on a larger dataset.
- Monitor the discriminator's average confidence on generated samples directly, not only the loss curves — per Section 4.3, a value persistently near zero is a direct, quantitative early warning of an overpowered discriminator, catchable before the sample-quality plateau becomes visually obvious.

11.1 A Note on Reproducibility

Per Day 28 §11.1 through Day 33 §11.1's reproducibility guidance, GAN training involves every source of randomness earlier days identified (weight initialization, data augmentation, Day 27 §6) plus a GAN-specific addition: the noise vector sampled as the generator's input is itself a further source of randomness

affecting which specific samples are produced and, per Section 4's discussion, potentially affecting whether a given training run encounters mode collapse or trains stably at all. For PP14's three-approach comparison specifically, fixing random seeds — including the noise-sampling seed, not only weight initialization — across all three training runs is worth doing explicitly, since GAN training's inherent instability (Section 3.2) makes run-to-run variance a larger practical concern here than for any single-network architecture earlier in this compendium.

11.2 A Note on Comparing Against Published Benchmarks

As with Day 31 §11.2, Day 32, and Day 33's benchmark notes, comparing PP14's own FID or qualitative results directly against published GAN benchmark figures is rarely meaningful without significant caveats. Published GAN results are typically computed on full-scale datasets with training runs spanning far more iterations and far more compute than a single Colab T4 session provides, and FID specifically is sensitive to the number of samples used to estimate it — a small-sample FID computed under PP14's compute constraints is not directly comparable to a published FID computed on tens of thousands of samples. The meaningful comparison for this assignment is always the three approaches against each other under your own matched conditions (Section 9.5), not any one approach against its own differently-resourced published benchmark result.

Glossary of Terms

This day has introduced a substantial amount of specialised vocabulary. The glossary below collects the most important terms in one place for quick reference, with a pointer back to the section where each is developed in full.

Nash equilibrium — The theoretical ideal GAN training endpoint at which the generator's samples are statistically indistinguishable from real data and the discriminator can do no better than chance at telling them apart (Section 2).

Generator — The GAN network that takes random noise (and, for conditional variants, a condition) as input and produces a synthetic image (Section 2).

Discriminator — The GAN network that classifies a given image as real (from training data) or fake (from the generator) (Section 2).

Minimax objective — The adversarial training formulation in which the discriminator maximizes its classification accuracy while the generator minimizes it, with no fixed external target for either network (Section 2).

Non-saturating loss — A reformulated generator objective that directly maximizes the discriminator's confidence in generated samples, avoiding the vanishing-gradient weakness of the original minimax generator loss (Section 3.4).

Mode collapse — A failure mode in which the generator produces a narrow range of outputs that reliably fool the current discriminator, sacrificing the diversity of the true data distribution (Section 4.1).

DCGAN guidelines — A set of architectural conventions (strided convolutions, batch normalization, no fully-connected layers, specific activation choices) that substantially stabilize GAN training (Section 5).

Conditional GAN (cGAN) — A GAN variant in which both generator and discriminator receive an additional conditioning input (e.g., a class label), enabling controllable generation (Section 6).

pix2pix — A conditional GAN for paired image-to-image translation, using a U-Net-style generator (Section 7.1).

CycleGAN — An unpaired image-to-image translation model using cycle-consistency loss as its training signal, requiring no aligned source-target pairs (Section 7.2).

Cycle-consistency loss — A self-supervised training signal requiring that translating an image to another domain and back should recover the original image (Section 7.2).

Progressive GAN — A training strategy that grows both networks' resolution gradually over training, avoiding the instability of training directly at high resolution (Section 8.1).

StyleGAN — A high-resolution GAN architecture injecting style information at every generator resolution level via an intermediate latent space, producing a disentangled, controllable latent representation (Section 8.2).

Fréchet Inception Distance (FID) — The standard generative evaluation metric, comparing the statistical distribution of deep features between real and generated images (Section 9.2).

Chapter Summary: Key Takeaways

- Generative modeling has no ground truth to compare against, fundamentally distinguishing it from every discriminative task in Days 26–33 (§1).
- A GAN trains two networks adversarially — a generator and a discriminator — via a minimax objective with no fixed external target for either (§2).
- Neither GAN loss curve alone signals training progress; both oscillate, and inspecting generated samples directly is the more reliable check (§3.2).
- Mode collapse, instability, and an overpowered discriminator are three distinct failure modes with distinct diagnostic signatures — correct diagnosis determines the right fix (§4).
- DCGAN's architectural guidelines (strided convolutions, batch norm, no dense layers) became the standard stabilized starting template for nearly all later GAN variants (§5).
- Conditional GANs add controllable generation by feeding a condition into both networks; image-to-image translation extends this to whole-image conditioning (§6–§7).
- pix2pix needs paired data and a U-Net generator; CycleGAN removes the pairing requirement via cycle-consistency, a self-supervised signal needing no ground truth (§7).

- Progressive growing and StyleGAN's disentangled latent space made high-resolution, controllable generation practical — and correspondingly raised the stakes of Section 10.2's ethics discussion (§8).
- FID is the current standard generative metric, directly comparing real and generated feature distributions; Inception Score and human evaluation each address gaps FID leaves open (§9).
- GANs (this day) and diffusion models (Day 35) are two structurally different answers to the same generative problem, trading training stability against generation speed (§10.1).

Assignment / Practical Project Brief

PP14: Image Generation with GANs

★ GRADED ASSIGNMENT

This is a graded assignment. The core requirement: implement all three approaches — vanilla/DCGAN, conditional GAN, and image-to-image translation — and directly compare them using §9.5's methodology, mirroring the parallel-implementation structure PP12, PP19, and PP15 each used for their own comparisons.

A Core Requirements

Implement all three approaches: (A) a vanilla/DCGAN unconditional generator trained from scratch, following §5's architectural guidelines; (B) a class-conditioned GAN (cGAN) per §6, demonstrating controllable generation on the same or a labeled variant of Approach A's dataset; and (C) an image-to-image translation model (pix2pix or CycleGAN, per §7) on a paired or unpaired image-pair dataset. Each approach should be genuinely trained, not merely run at inference from a pretrained checkpoint.

B Dataset

Choose Colab T4-appropriate data given GAN training's compute cost: a compact, low-resolution dataset (a modest number of classes, small image size) for Approaches A and B, and a small paired or unpaired image-pair dataset for Approach C. Keeping resolution modest (per §8's discussion of why naive high-resolution training is unstable) is a reasonable and expected scope decision for this assignment, not a shortcut to be apologized for.

C Deliverables

- Three working, trained pipelines (vanilla/DCGAN, conditional GAN, image-to-image translation) with documented training configurations

- Evaluation with FID where practical, or structured qualitative comparison where FID is impractical given compute budget (§9.5), applied consistently across all three approaches
- Documented training stability observations for each approach against §4's failure modes (which, if any, were encountered, and how they were diagnosed and addressed)
- A comparison table/plot: sample quality, training stability, controllability, and compute cost across the three approaches
- A short written recommendation for when each approach fits a given generation task, tying observed results back to Sections 5–7's architectural tradeoffs rather than reporting outputs alone

D Grading Rubric

As with PP11, PP15, and PP19's comparisons, evaluation rigor and comparison quality together carry the largest share of the grade — a submission with three technically working but sloppily-compared models will score notably lower than one with three adequately-trained models compared with real methodological care, honest reporting of any failure modes encountered, and a genuine, dataset-grounded recommendation.

E Tips and Common Pitfalls

- Mode collapse (§4.1), training instability (§4.2), and an overpowered discriminator (§4.3) are all plausible outcomes on a compact, single-session dataset — budget time to diagnose and respond to at least one of them, per this section's Common Pitfalls checklist.
- Verify the non-saturating generator loss (§3.4) is implemented correctly before spending significant training time debugging an apparently stalled generator.
- For CycleGAN specifically, tune the cycle-consistency loss weight deliberately (§7.2) — the default value from a published implementation may not transfer cleanly to a smaller, different dataset.
- Restart the Colab runtime (or explicitly free GPU memory) between training the three approaches if all are trained in the same notebook session, echoing PP11 and PP15's identical GPU-memory tip.
- Save generated sample grids at regular checkpoints throughout training, not only at the end — per §3.2, this is the most reliable way to catch mode collapse or instability early enough to still respond to it within the assignment's time budget.

F Submission Checklist

☐ Vanilla/DCGAN approach trained and evaluated, following §5's architectural guidelines

☐ Conditional GAN approach trained and evaluated, with controllability demonstrated against the unconditional baseline

- ☐ Image-to-image translation approach (pix2pix or CycleGAN) trained and evaluated
- ☐ FID computed where practical, or a consistent qualitative comparison protocol applied across all three approaches otherwise
- ☐ Training stability observations documented against §4's failure modes for each approach
- ☐ Comparison table/plot completed with your own measured results, not placeholders
- ☐ Written recommendation explicitly connects results to Sections 5–7's architectural tradeoffs
- ☐ Code is documented, and a short README explains how to reproduce your results

End of Day 34 — ready for gap review and expansion into full compendium.